CEUB

PLANEJAMENTO LÓGICO

TASKTRACKER

Bootcamp II — Etapa Inicial

Paulo Henrique de Melo Bezerra

Ciência da Computação

Brasília — DF
2026

1. Apresentação e Entendimento do Problema

1.1 Identificação

Nome	Paulo Henrique de Melo Bezerra
Curso	Ciência da Computação
Polo/Turma	Campus Asa Norte — Noturno — Turma única
E-mail institucional	paulo.henriqueb@sempreceub.com

1.2 O Problema a Ser Resolvido

Atualmente, muitas pessoas precisam realizar diversas tarefas durante o dia, seja em atividades acadêmicas, profissionais ou pessoais. Quando essas tarefas não são organizadas de maneira adequada, pode ser difícil lembrar o que precisa ser feito, identificar quais atividades são mais importantes e acompanhar aquilo que já foi concluído.

O problema que o sistema pretende resolver é justamente essa dificuldade de organização. Sem uma ferramenta adequada, uma pessoa pode esquecer tarefas, perder prazos ou não conseguir visualizar claramente quais atividades ainda estão pendentes.

Portanto, existe a necessidade de uma forma simples e organizada de registrar e acompanhar tarefas, permitindo que o usuário tenha uma visão mais clara de suas responsabilidades.

1.3 A Solução Proposta — TaskTracker

Como solução para esse problema, será planejado o TaskTracker, um sistema voltado para o gerenciamento e a organização de tarefas.

O sistema permitirá que o usuário cadastre suas tarefas e registre informações importantes sobre elas, facilitando sua organização. Dessa forma, será possível visualizar as tarefas cadastradas, identificar suas prioridades e acompanhar quais atividades ainda precisam ser realizadas.

O objetivo do TaskTracker é oferecer uma maneira simples e organizada de controlar tarefas, ajudando o usuário a visualizar melhor suas atividades e acompanhar seu progresso.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

Para que o TaskTracker consiga cadastrar e organizar as tarefas, o usuário deverá fornecer algumas informações ao sistema.

- **Título da tarefa:** nome utilizado para identificar a atividade.
- **Descrição:** informações adicionais sobre o que precisa ser realizado.
- **Prioridade:** nível de importância da tarefa, podendo ser Alta, Média ou Baixa.
- **Data limite:** prazo definido para a conclusão da tarefa.

Além dessas informações, durante a utilização do sistema o usuário também poderá indicar uma tarefa que deseja visualizar ou marcar como concluída.

2.2 Saídas de Dados

Após receber e processar as informações fornecidas pelo usuário, o TaskTracker deverá apresentar informações que permitam acompanhar as tarefas cadastradas.

- **Lista de tarefas cadastradas:** apresentação das tarefas e de suas principais informações.
- **Tarefas pendentes:** identificação das atividades que ainda precisam ser realizadas.
- **Confirmação de cadastro:** mensagem informando que uma nova tarefa foi cadastrada com sucesso.

- **Confirmação de conclusão:** mensagem apresentada quando uma tarefa é marcada como concluída.
- **Total de tarefas concluídas:** quantidade de tarefas que já foram finalizadas.
- **Mensagens de erro ou aviso:** informações apresentadas quando algum dado informado não é válido.

2.3 Regras do Sistema

Para garantir o funcionamento correto do TaskTracker, o sistema deverá seguir as regras abaixo:

1. Uma tarefa não poderá ser cadastrada sem um título.
2. A prioridade de uma tarefa deverá ser obrigatoriamente classificada como Alta, Média ou Baixa.
3. Toda nova tarefa cadastrada deverá inicialmente ser considerada pendente, até que o usuário indique sua conclusão.
4. Uma tarefa somente poderá ser marcada como concluída caso esteja cadastrada no sistema.
5. Caso o usuário forneça alguma informação inválida, o sistema deverá informar o problema e solicitar uma nova entrada, evitando o cadastro incorreto.

3. Receita Lógica do Sistema (Algoritmo em Português)

> A lógica foi descrita em linguagem simples e sequencial para representar como o sistema deverá funcionar antes da implementação em Python.

3.1 Passo a Passo do Cadastro de uma Tarefa

1. Solicitar ao usuário o título da tarefa.
2. Verificar se o título informado não está vazio.
3. Caso esteja vazio, informar o erro e solicitar novamente o título.
4. Solicitar a descrição da tarefa.
5. Solicitar a prioridade da tarefa, podendo ser Alta, Média ou Baixa.
6. Verificar se a prioridade informada é válida.
7. Caso seja inválida, avisar o usuário e solicitar novamente a prioridade.
8. Solicitar a data limite da tarefa.
9. Registrar a tarefa inicialmente com o status de pendente.
10. Salvar a tarefa na lista de tarefas.
11. Exibir uma mensagem informando que a tarefa foi cadastrada com sucesso.

3.2 Passo a Passo da Visualização das Tarefas

1. Acessar a lista de tarefas cadastradas.
2. Verificar se existe alguma tarefa registrada.
3. Caso a lista esteja vazia, informar ao usuário que nenhuma tarefa foi cadastrada.

4. Caso existam tarefas, apresentá-las uma por uma na tela.
5. Mostrar o título, descrição, prioridade, data limite e situação de cada tarefa.

3.3 Passo a Passo para Concluir uma Tarefa

1. Apresentar as tarefas cadastradas.
2. Solicitar ao usuário qual tarefa deseja marcar como concluída.
3. Verificar se a tarefa informada existe.
4. Caso não exista, informar o erro ao usuário.
5. Caso exista, alterar sua situação de pendente para concluída.
6. Exibir uma mensagem confirmando que a tarefa foi concluída com sucesso.

3.4 Fluxo Geral do Sistema

1. Iniciar o TaskTracker.
2. Apresentar as funcionalidades disponíveis ao usuário.
3. Permitir que o usuário escolha entre cadastrar, visualizar ou concluir uma tarefa.
4. Executar a funcionalidade escolhida.
5. Após a execução, permitir que o usuário escolha uma nova opção.
6. Caso seja informada uma opção inválida, exibir uma mensagem de erro.
7. Repetir o processo até que o usuário escolha encerrar o programa.
8. Encerrar o TaskTracker.

4. Mapeamento Arquitetural para Git e GitHub

4.1 Desenho da Árvore do Repositório

A estrutura do repositório foi planejada para manter o projeto organizado e facilitar seu desenvolvimento nas próximas fases.

```
tasktracker-projeto/
├── README.md
├── .gitignore
├── docs/
│   └── planejamento_logico.pdf
└── src/
    └── main.py
```

- **README.md:** será utilizado para apresentar o projeto no GitHub, explicando seu objetivo e principais informações.
- **.gitignore:** será utilizado para definir arquivos temporários ou desnecessários que não deverão ser enviados ao repositório.
- **docs/:** pasta destinada à documentação do projeto.

- **planejamento_logico.pdf:** arquivo contendo o planejamento desenvolvido nesta etapa inicial.
- **src/:** pasta destinada aos arquivos de código-fonte do sistema.
- **main.py:** será o arquivo principal do programa em Python.

4.2 Plano de Aprendizado para Git e GitHub

Na Fase 2, espero aprender a utilizar melhor o Git e o GitHub para controlar e organizar as diferentes versões do projeto. Quero entender como funcionam os commits e como registrar as alterações realizadas de forma organizada, além de aprender a criar branches para desenvolver novas funcionalidades sem afetar diretamente a versão principal do sistema. Também espero aprender a resolver possíveis conflitos entre alterações, acompanhar o histórico de versões e publicar corretamente o projeto no GitHub.

5. Link da Apresentação em Vídeo

O vídeo de apresentação será publicado no YouTube e apresentará, de forma resumida, o problema identificado, a solução proposta, a receita lógica do sistema e o planejamento para as próximas fases.

Link: https://youtu.be/0oST70-UeDc